

RentTitan MVP

Hamdi Adam Abouleila Selim, Robert Anaïs

PROJECT IDEA

RentTitan helps tenants create a clear rental dossier and receive a dossier strength score out of 100. The MVP uses manual input fields and labeled document upload slots, then calculates a transparent rule-based score with improvement suggestions.

MVP goal Google login -> create dossier -> upload dummy documents -> calculate score -> show breakdown.	Core decision No automatic PDF reading in the MVP. Users choose the document type before uploading.
Architecture React frontend, Node main backend, Document Service, Scoring Service, PostgreSQL, MongoDB.	Presentation focus Show authentication, services, databases, REST + GraphQL, and JWT-protected API demo.

React/Vite	Node/Express	PostgreSQL	MongoDB	JWT + Google OAuth
-------------------	---------------------	-------------------	----------------	---------------------------

1. MVP Scope Required to Pass

The MVP is intentionally small: build only the features needed to satisfy the full-stack, microservice, database, authentication, API, and presentation requirements.

Requirement	RentTitan MVP implementation
Full-stack JS app	React/Vite frontend + Node.js/Express backend projects.
Main backend + microservices	Main Backend plus two separate microservice projects: Document Service and Scoring Service.
At least 2 databases	PostgreSQL for users/dossiers/scores; MongoDB for document metadata/checklists.
Google login + protected routes	Google OAuth login creates JWT; protected pages redirect to login if unauthenticated.
API as a Service	Protected REST endpoint returns dossier score only when a valid JWT is passed.
JWT demo	Postman call with JWT succeeds; same call without JWT returns access denied.
At least 2 API paradigms	REST for CRUD and protected API demo; GraphQL for dashboard score queries.
External API bonus	Optional LLM API can rewrite advice in natural language; score remains rule-based.
Presentation readiness	Demo app, tech stack, service boundaries, databases, APIs, authentication flow.

In Scope vs. Out of Scope

Build for MVP	Do not build yet
• Google login and JWT session • Dossier profile form • Labeled upload slots with checklist • Rule-based score /100 • REST + GraphQL APIs • Postman JWT/no-JWT demo	• OCR or automatic PDF extraction • Real document verification • Landlord marketplace • Payment system • Mobile app • Cloud deployment unless teacher confirms

2. Architecture Overview

RentTitan will be split into one main backend and two independent microservice projects. The main backend acts as the secure entry point for the frontend and coordinates service calls.

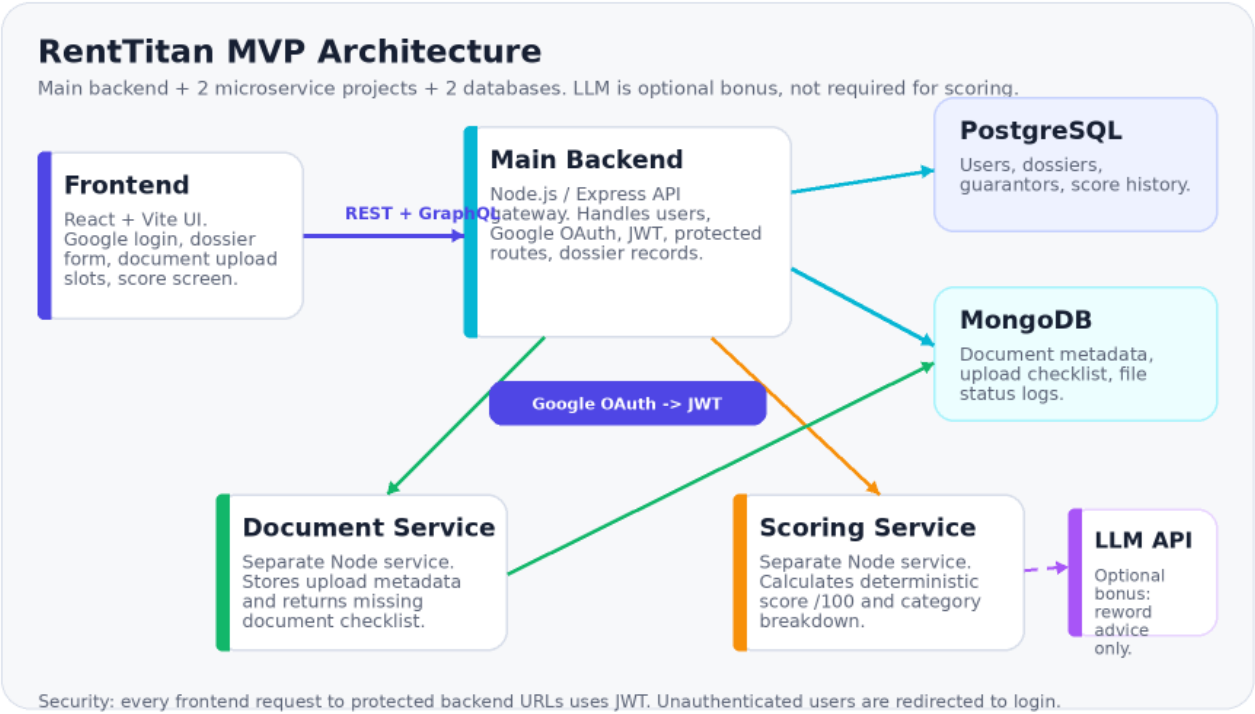


Diagram 1 - Minimal architecture. The LLM API is optional and only improves wording; the score itself is deterministic.

Component	Tech	Responsibility	Data
Frontend	React + Vite	Google login, forms, upload UI, score dashboard	-
Main Backend	Node.js + Express	Auth gateway, dossiers, protected API, GraphQL endpoint	PostgreSQL
Document Service	Node.js + Express	Upload slots, file metadata, missing document checklist	MongoDB
Scoring Service	Node.js + Express	Deterministic score /100, breakdown, improvement flags	PostgreSQL
LLM Advice API	External API / optional	Human-friendly explanation only; not score authority	No app DB

3. Scoring Model and API Design

The score is calculated from structured values entered by the user plus the document checklist returned by the Document Service. This keeps the MVP explainable and easy to defend.

Category	Pts	Rule in MVP
Document completeness	25	Required slots uploaded: ID, proof of income/status, proof of address, guarantor ID, guarantor income/tax proof.
Rent affordability	25	$\text{targetRent} / \text{monthlyIncome}$. Lower ratio earns more points; high ratio creates warning.
Guarantor strength	25	Guarantor income relative to rent + guarantor documents present.
Profile stability	15	CDI/alternance/student status, income reliability, clear situation.
Profile clarity	10	Complete contact fields, consistent inputs, explained income/guarantor relationship.
Total	100	Score level: Excellent, Strong, Average, Weak, Very weak.

Example Output

Score: 72 / 100

Level: Average dossier

Top fix: add guarantor tax notice and proof of income.

Important implementation rule

The app does not need to read PDFs. The user enters income/rent/guarantor data manually and uploads files into named slots. The Document Service only checks which slots are filled.

API Paradigms

Paradigm	Endpoint / query	Purpose
REST	POST /dossiers	Create or update dossier input fields.
REST	POST /documents/upload	Upload dummy file into a named document slot.
REST + JWT	GET /api/v1/dossier-score/:id	Public API-as-a-service demo; requires Bearer token.

GraphQL	query myDossier { score missingDocuments suggestions }	Dashboard query showing second API paradigm.
Error case	No JWT / invalid JWT	Return 401 { error: "Access denied" }.

4. Build Order and Demo Plan

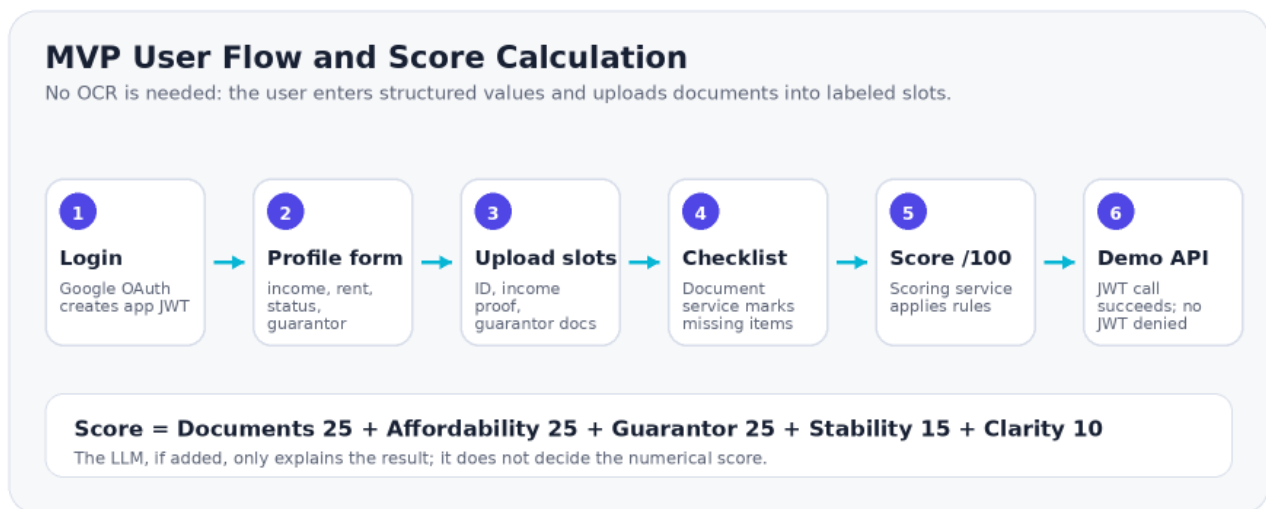


Diagram 2 - End-to-end MVP flow from login to score and JWT-protected API demo.

#	Milestone	Definition of done
1	Initialize repos	frontend, main-backend, document-service, scoring-service, docker-compose.
2	Auth first	Google OAuth login, JWT middleware, protected route redirect.
3	Dossier form	manual inputs: income, rent, status, guarantor data.
4	Documents	labeled upload slots + missing checklist; no OCR needed.
5	Score engine	rules, category breakdown, score history, suggestions.
6	APIs	REST CRUD, protected API-as-service, GraphQL dashboard query.
7	Demo polish	seed data, dummy PDFs, Postman JWT/no-JWT calls, 12-minute script.

Final Demo Checklist

App demo

- ☐ Login with Google
- ☐ Create dossier with manual fields
- ☐ Upload dummy documents into slots
- ☐ Show score breakdown and suggestions

Technical proof

- ☐ Show service folders/projects
- ☐ Show PostgreSQL and MongoDB data
- ☐ Show REST + GraphQL usage
- ☐ Postman with JWT succeeds; no JWT denied